

Guldkortet utvärdering

Generellt var Guldkortet en väldigt utmanande uppgift. Jag stötte på många problem under projektets gång, och det var ofta svårt att identifiera de buggar som uppstod samt deras grundorsaker. På grund av detta har konstruktionen av programmet varit mycket mer tidskrävande än jag förväntade mig, men jag har erhållit många värdefulla lärdomar längs vägen.

Vad som fungerade bra

Precis som jag påpekade i min planering lade jag stor vikt vid konkret felhantering av olika användningsfall. Programmet uppfyller nu detta genom en egen, dedikerad exception-hierarki ([GuldkortetException](#) som basklass för bland annat [InvalidCodeFormatException](#), [DatabaseConnectionException](#) och [CardAlreadyUsedException](#)), med tydlig text som beskriver varje situation för användaren. Jag har övervägt flera olika felaktiga utfall, till exempel om NOS_Export skickar ett felaktigt formaterat meddelande, en okänd användar-ID, en okänd kort-ID, eller ett kort som redan lösts in.

Jag byggde också en egen generisk listklass ([MyRewardList](#)) baserad på en array med dynamisk storleksändring, istället för att använda .NET:s inbyggda [List<T>](#), för att visa en djupare förståelse för hur sådana datastrukturer fungerar under ytan. Utöver [Reward](#)-hierarkin ([ThunderCat](#), [CrystalSteed](#), [FlyingSquirrel](#), [FireHound](#)) resulterade min satsning på felhantering även i en annan separat arvsstruktur i projektet.

Avvikelser från planeringen

Det finns några delar där mitt arbete inte helt följt den ursprungliga planeringen. Jag bytte databasmotor från SQL Server till SQLite, efter att de utdelade [.mdf](#)-filerna visade sig vara inkompatibla med min lokala LocalDB-version. Det här matchade faktiskt min ursprungliga plan bättre, eftersom SQLite var tänkt att användas redan från början. Jag valde också att ha [Email](#) istället för [Kommun](#) som attribut i kundregistret, ett medvetet designval för att bättre spegla verklig kontaktinformation.

Jag lade även till fler egenskaper på de underklasser som ärver från [Reward](#)-klassen (t.ex. [Speed](#), [Element](#) och [Rarity](#) på [CrystalSteed](#)) för att tydligare visa ett exempel på polymorfism, utöver den enklare [GenerateMessage\(\)](#)-overriden.

Den mest komplexa buggen

Den mest komplexa buggen jag stötte på var ett kompatibilitetsproblem med SQLites native-bibliotek ([e_sqlite3.dll](#)), som gjorde att programmet kraschade så fort en

databasanslutning skulle öppnas. Genom att byta projektets målplattform från "Any CPU" till x64, samt säkerställa att rätt native-fil kopierades till build-mappen, kunde jag till slut lösa problemet. En annan ovanligt svårupptäckt bugg var en encoding-mismatch mellan min [StreamReader](#) och det externa programmet NOS_Export. NOS_Export skickade sin data i UTF-16, medan min kod förväntade sig ASCII (som först ändrades från UTF8), vilket resulterade i att bara halva varje meddelande lästes in korrekt. Jag löste detta genom att undersöka de faktiska byte-värdena i den mottagna datan, vilket avslöjade mönstret av "dolda" nolltecken mellan varje riktigt tecken.

Eget initiativ

Jag har även försökt implementera mina tidigare kunskaper från JavaScript/TypeScript för att bygga ett lite mer avancerat program än vad som förväntades av uppgiften.

Om jag hade haft mer tid

Om jag hade haft mer tid hade jag velat implementera en [PictureBox](#) i gränssnittet för att visuellt visa belöningens bild, istället för att bara lagra sökvägen som text i [ImagePath](#). Jag hade också velat utöka loggningen så att misslyckade kund-sökningar (inte bara kort-sökningar) registreras i [log.txt](#), för fullständig spårbarhet.